

4DGS行文逻辑

Abstract

I. Introduction

1. 应用需求：VR/AR需渲染动态场景，要求高帧率和低功耗

随着AR/VR等边缘端渲染设备的流行，静态场景已经无法模拟真实的场景特点，而动态场景由于在静态场景的基础上增加了时间维度，从而可以更完善地表征所有现实场景。动态场景的渲染复杂度更高，造成更高的延时和功耗，而这与边缘端硬件的高帧率、低功耗需求相违背。

2. 算法流行：4DGS是动态场景渲染sota算法，相比3DGS新增形变场计算

动态场景的表征与渲染近年来也有新兴的方案，例如nerf，4DGS等。虽然nerf具有较小的模型体量与较高的渲染质量，但是具有隐式表征和计算量大、帧率低的缺点。而4DGS以其显式的几何表征、可微的数学特性以及较高的渲染帧率，成为当下的热点，并且相对来讲更适合在边缘端部署。

3. 具体挑战：边缘GPU帧率低，因为形变、光栅化阶段存在大量冗余计算

虽然4DGS在桌面级显卡的渲染帧率很高（100fps以上），但是在边缘端GPU上的帧率往往达不到20fps（即使是Jetson orin），距离人眼的感知极限60Hz还有很大差距，这对于VR/AR等边缘端应用来说，是远远不够的。本文就将针对这个挑战，进行深入的分析，重点优化了耗时最长的形变、光栅化这两个阶段，找出了不必要的计算，并提出了软硬件协同设计的方案，使得4DGS能在低功耗下达到高帧率，从而适应边缘端场景部署。

4. 本文贡献

- 深入分析了4DGS算法的瓶颈和冗余计算的部分，通过分析的数据支持了我们的几个重要发现。
- 创新地提出了一种后训练的motion-aware标记法，将高斯球按照整个时间线上的形变程度进行分类，并且重构了数据流，按类分别跳过了不必要的形变场计算
- 提出了负载分配优化的foveated rendering及调度方案，人眼几乎无感的前提下，降低了光栅化的计算量，并且处理了其带来的负载不平衡的问题
- 设计与算法紧密配合的硬件架构，包括unfied预处理模块以及多分辨率光栅化模块，并且通过完善的实验数据证明了整个方案可以在低功耗下达到高帧率，满足边缘端渲染动态场景的需求。
(TODO)

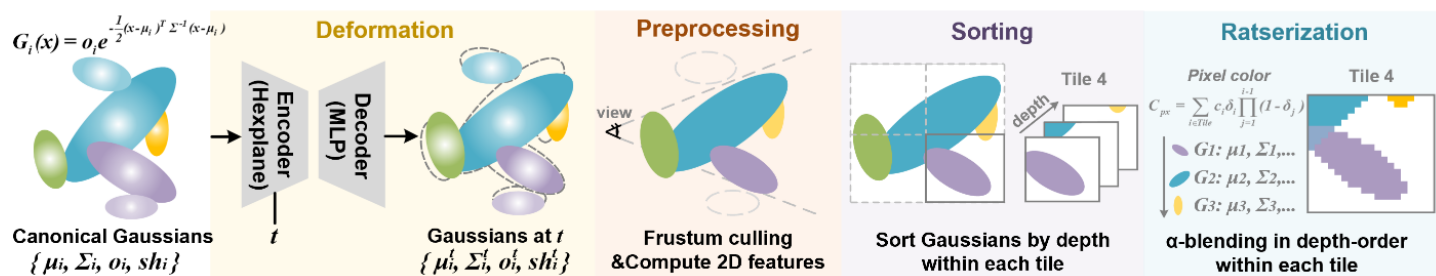
II. Background

1. 4DGS算法介绍

4DGS相较于3DGS增加了时间这个维度。为了表征一个动态场景，如果将其按照时间切片成多个帧，并对每一帧训练一个3DGS模型，其训练和存储成本是不可接受的。因此为了更高效地表征动态场景，通常是在一个由3DGS表示的静态场景的基础上，增加了一个能表征不同时间步下任意一个高斯球修正量的模块。或者说，3DGS是一套高斯球参数的集合，与时间无关；而4DGS则是一套高斯球，每个高斯球的参数都是时间的函数。为了达成这一点，有两种主流的方法，一种是将每个高斯球的参数进一步建模为时间的高斯分布；还有一种是通过一个神经网络来编解码每个高斯球在任意时间步下的形变。后者由于更小的模型size和更高的帧率，是本文研究的对象。

需要强调的一点是，对于一个特定的场景，都需要训练一套高斯球来表征该场景，包括了canonical高斯球以及配套的形变场hexplane和mlp。训练完成后，这一套高斯球就可以用于渲染。因此这也类似于预训练+推理的范式。

计算流程（如图）：算各高斯球在相机坐标系中xyz，算形变场（hexplane+mlp），视锥剔除，算协方差，排序，分块光栅化。



2. 4DGS硬件加速（工作较少，转而介绍一些3DGS加速）

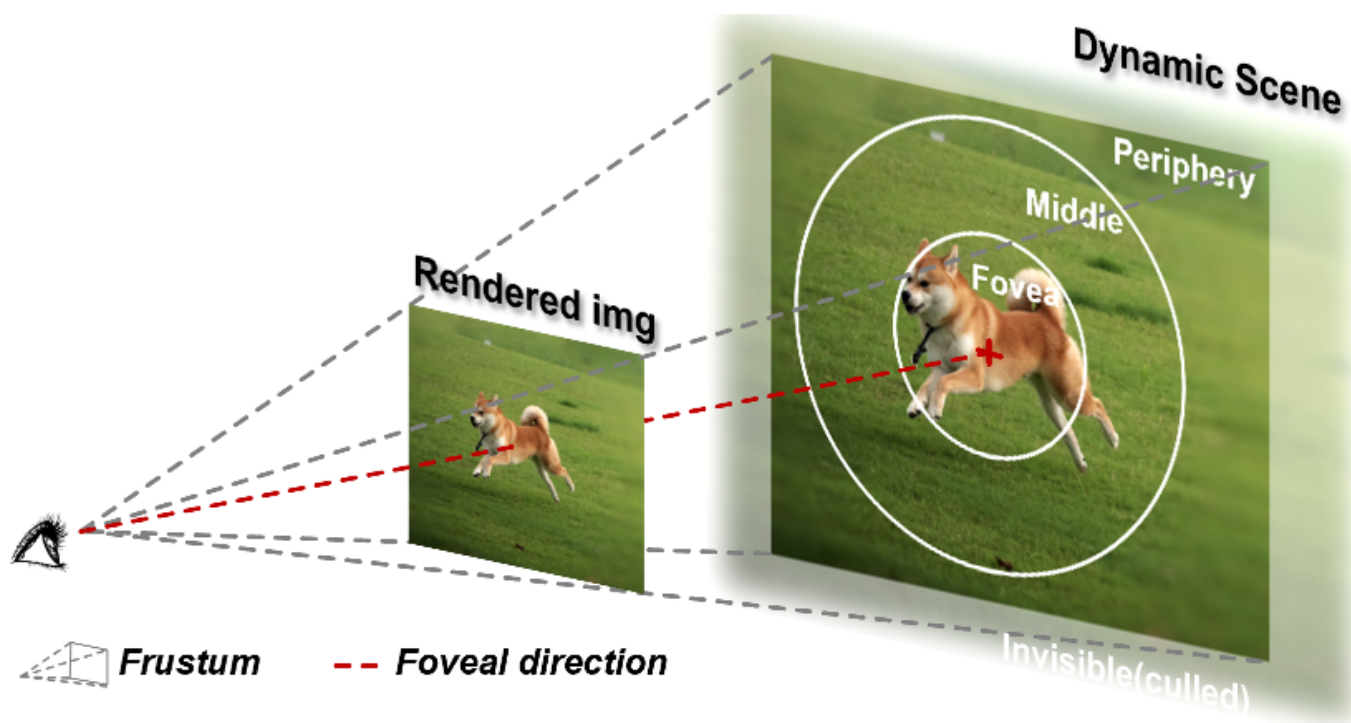
year	confere	title	catg
2026	ASPLOS	Neo: Real-Time On-Device 3D Gaussian Splatting with Reuse-and-Update Sorting Accel	ASIC
2026	ISSCC	A 1286fps 0.39mJ/Frame Modeling/Rendering Unified 3D GS Processor with Locality-Op	ASIC
2026	ISSCC	A 0.24mJ/Frame Quadratic Interpolation 4DGS Processor with Recursive Computation F	ASIC
2025	ASP-DAC	GSNorm: An Efficient 3D Gaussian Rendering Accelerator with Splat Normalization and	ASIC
2025	ASPLOS	ARC: Warp-level Adaptive Atomic Reduction in GPUs to Accelerate Differentiable Render	GPU
2025	DAC	GS-TG: 3D Gaussian Splatting Accelerator with Tile Grouping for Reducing Redundant S	ASIC
2025	DAC	STREAMINGGS: Voxel-Based Streaming 3D Gaussian Splatting with Memory Optimizatio	ASIC
2025	DAC	GauRast: Enhancing GPU Triangle Rasterizers to Accelerate 3D Gaussian Splatting	GPU
2025	DAC	Local-GS: An Order-Independent Gaussian Splatting Training Accelerator Exploiting Spl	ASIC
2025	DAC	GSAcc: Accelerate 3D Gaussian Splatting via Depth Speculation and Gaussian-centric R	ASIC
2025	DATE	PS-GS: Group-Wise Parallel Rendering with Stage-Wise Complexity Reductions for Real-	ASIC
2025	DATE	FAMERS: An FPGA Accelerator for Memory-Efficient Edge-Rendered 3D Gaussian Splatti	FPGA
2025	HPCA	VR-Pipe: Streamlining Hardware Graphics Pipeline for Volume Rendering	GPU
2025	HPCA	Gaussian Blending Unit: An Edge GPU Plug-in for Real-Time Gaussian-Based Rendering	GPU
2025	HPCA	GSArch: Breaking Memory Barriers in 3D Gaussian Splatting Training via Architectural S	ASIC
2025	ICCAD	No Redundancy, No Stall: Lightweight Streaming 3D Gaussian Splatting for Real-time R	ASIC
2025	ICCAD	GauPRE: A Pattern-based Rendering Engine for Gaussian Splatting on Edge Device	GPU
2025	ISCAS	HyperGS: Efficient Real-Time 3D Gaussian Rendering Processor Through Hierarchical Sc	ASIC
2025	ISCAS	ScatterSplatting: Enhanced View Synthesis in Scattering Scenarios via Joint NeRF and C	ASIC
2025	ISSCC	1.78mJ/Frame 373fps 3D GS Processor Based on Shape-Aware Hybrid Architecture Usin	ASIC
2025	ISSCC	IRIS: A 8.55mJ/frame Spatial Computing SoC for Interactable Rendering and Surface-Av	ASIC
2025	arxiv	3DGauCIM: Accelerating Static/Dynamic 3D Gaussian Splatting via Digital CIM for High F	ASIC
2025	MICRO	GCC: A 3DGS Inference Architecture with Gaussian-Wise and Cross-Stage Conditional Pr	ASIC
2024	A-SSCC	A 66.6 FPS High Quality Gaussian Splats Rendering FPGA Processor with Reconfigurable	FPGA
2024	ASPLOS	GSCore: Efficient Radiance Field Rendering via Architectural Support for 3D Gaussian Sp	ASIC

3. 中央凹渲染（foveated rendering）

论文标题	出版期刊/会议	被引量	第一单位
Process Only Where You Look: Hardware and Algorithm Co-optimization for Efficient Gaze-Tracked Foveated Rendering in Virtual Reality	ISCA2025	暂无	纽约大学 (NYU)
A3FR: Agile 3D Gaussian Splatting with Incremental Gaze Tracked Foveated Rendering in Virtual Reality	ACM International Conference on Supercomputing	新论文，暂无	需进一步确认

VR-Splatting: Foveated Radiance Field Rendering via 3D Gaussian Splatting and Neural Points	arXiv (2410.17932)	暂无	德国埃朗根-纽伦堡大学
VRSplat: Fast and Robust Gaussian Splatting for Virtual Reality	arXiv (2505.10144)	新论文，暂无	需进一步确认
Fov-GS: Foveated 3D Gaussian Splatting for Dynamic Scenes	IEEE Transactions on Visualization and Computer Graphics	暂无	北卡罗来纳大学教堂山分校
FlashGS: Efficient 3D Gaussian Splatting for Large-scale and High-resolution Rendering	CVPR 2025	新论文，暂无	需进一步确认
Foveated rendering: A state-of-the-art survey	Computational Visual Media, 2023, 9(2):195-228	暂无	北京航空航天大学
Towards foveated rendering for gaze-tracked virtual reality	ACM Transactions on Graphics, 2016	暂无	NVIDIA
Foveated 3D graphics	ACM SIGGRAPH Asia 2012	暂无	微软研究院

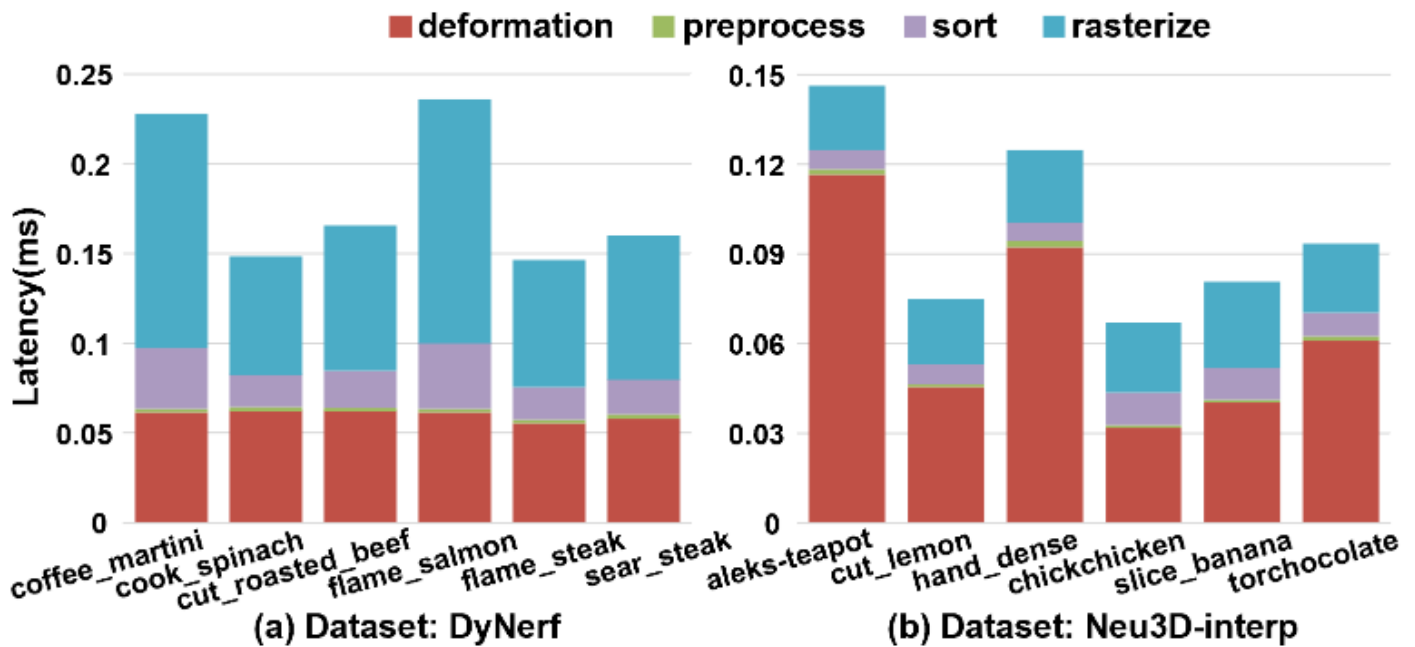
人类视觉系统的最小视角分辨率（Minimum Angle of Resolution, MAR）线性衰减模型。根据 **Guenter et al. (2012)** 的用户研究，MAR随着偏心率（eccentricity）呈线性增长，斜率 m 约为 $1.32 - 1.65 \text{ arcmin/degree}$ 。在 30° 的偏心率下，人眼的分辨能力下降了约10倍以上。



III Motivation&Characterization

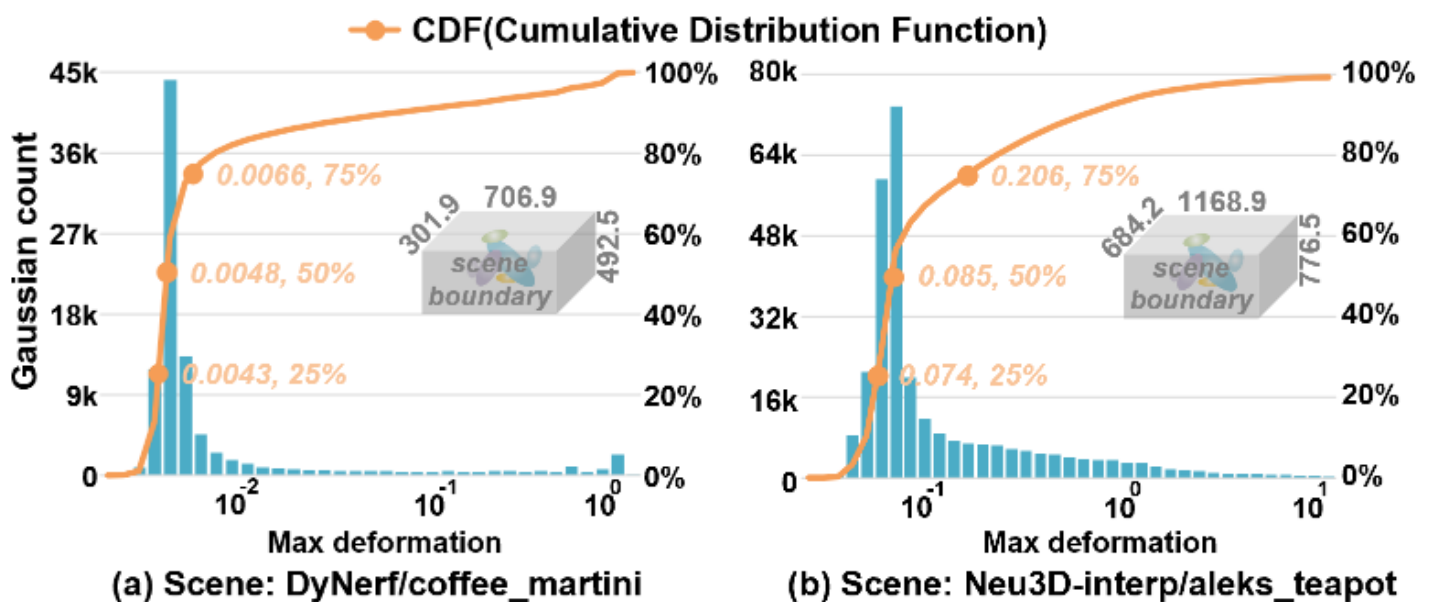
1. Latency breakdown --> 发现1: 形变场、光栅化占比高，应重点优化

数据支持：dynerf数据集和hypernerf数据集中各场景渲染延时分析证明预处理中的形变计算占据30%甚至超过50%（取决于高斯球数量）。使用jetson agx orin实验得到数据。



2. 高斯球静止程度分析 --> 发现2: 大多高斯形变极小，形变场计算可以跳过

数据支持：统计了两个场景高斯球在整个时间跨度内最大的坐标形变，绝大多数都极小

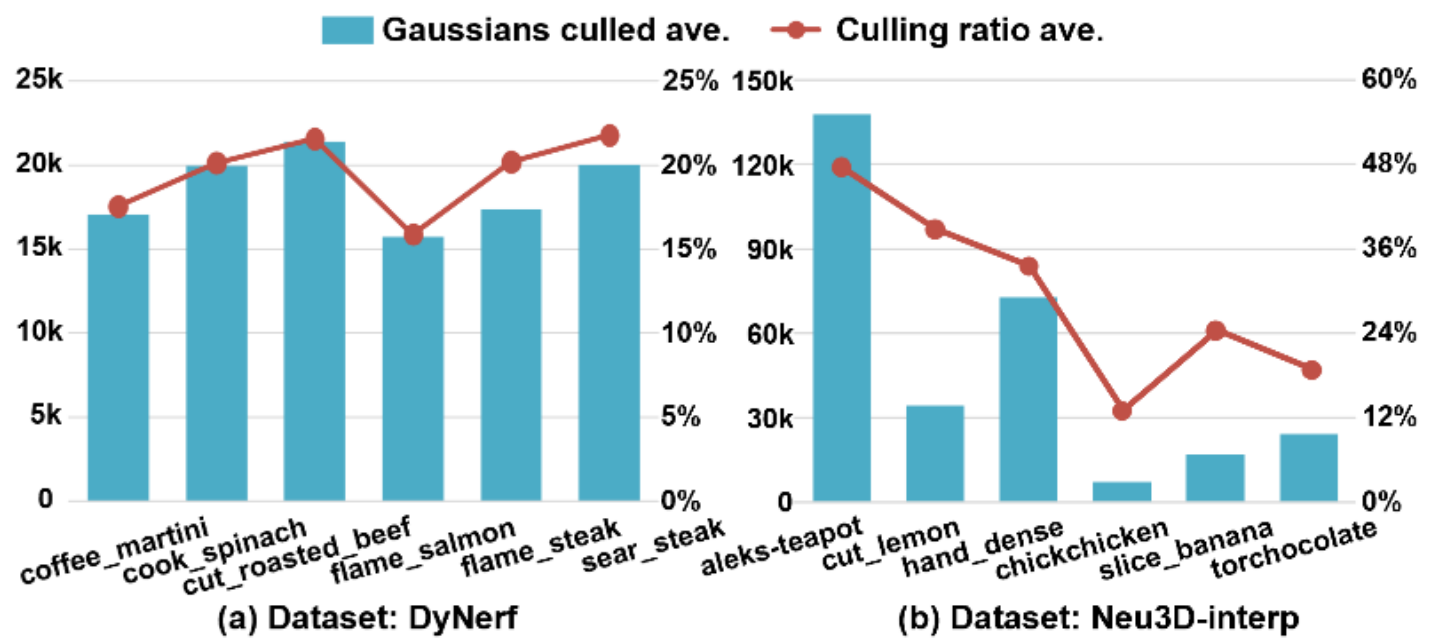


3. 视锥剔除率分析 --> 发现3: 视锥剔除阶段大量高斯被剔除，其形变场计算可以跳过

数据支持：统计dynerf数据集和hypernerf数据集中各场景在视锥剔除阶段剔除掉的高斯球比例，可以发现较大一部分高斯球将被剔除，而他们被剔除前的形变计算将是浪费的。考虑到上一个发现证明了整个时间跨度里大多高斯球的形变极小，因此可以近似使用未形变前的坐标来计算视锥剔除。

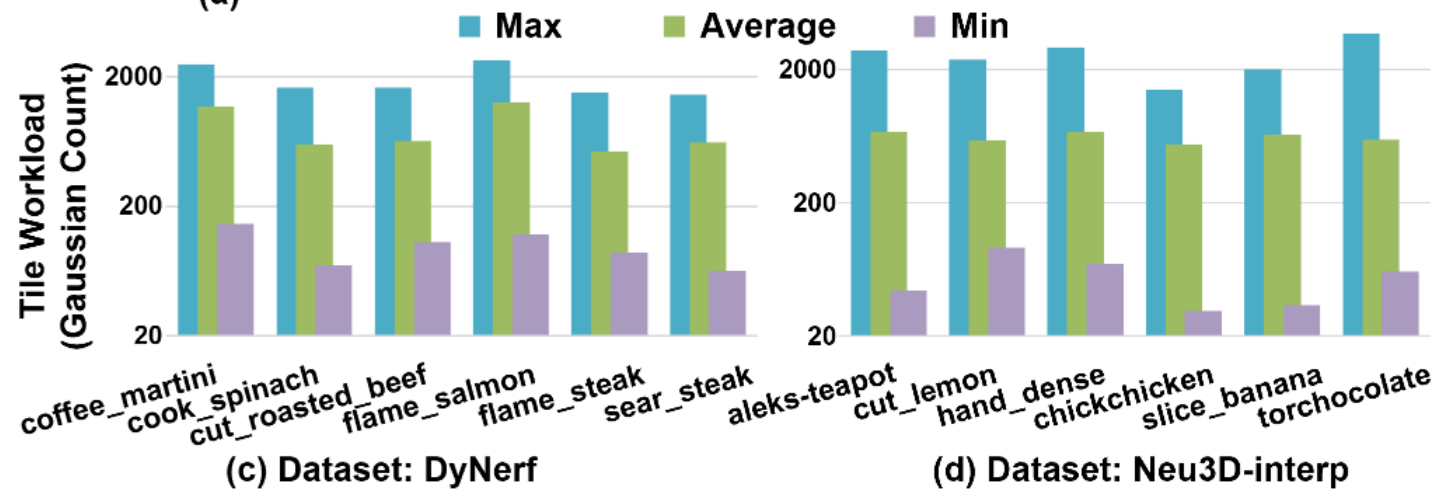
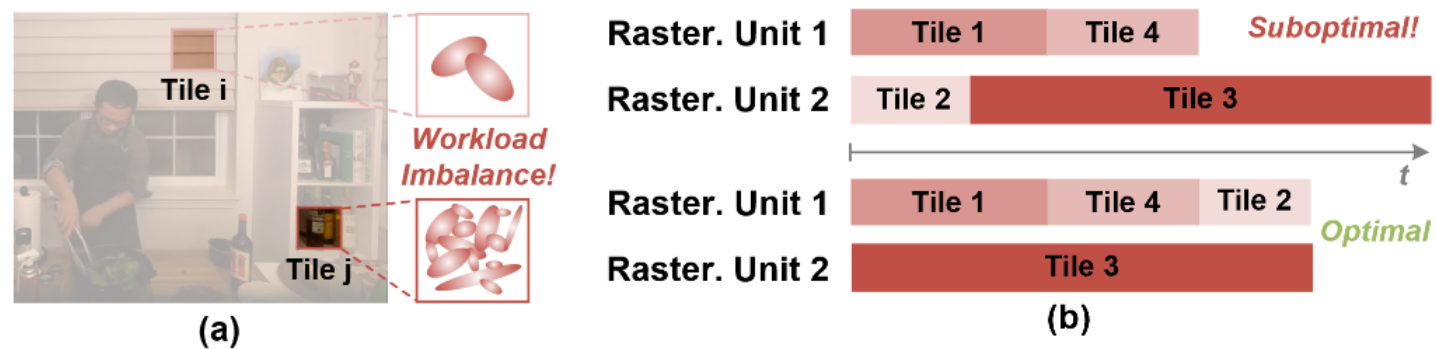
原算法：先做形变计算，再做视锥剔除

现在：先做视锥剔除，再做形变计算



4. tile负载（中央凹渲染）分析--> 发现4: 不同tile的负载差距悬殊，贪心分配导致利用率低

数据支持：dynerf和hypernerf数据集中各场景中（test split, t=0）tile需要处理的高斯球（即工作负载）差距悬殊

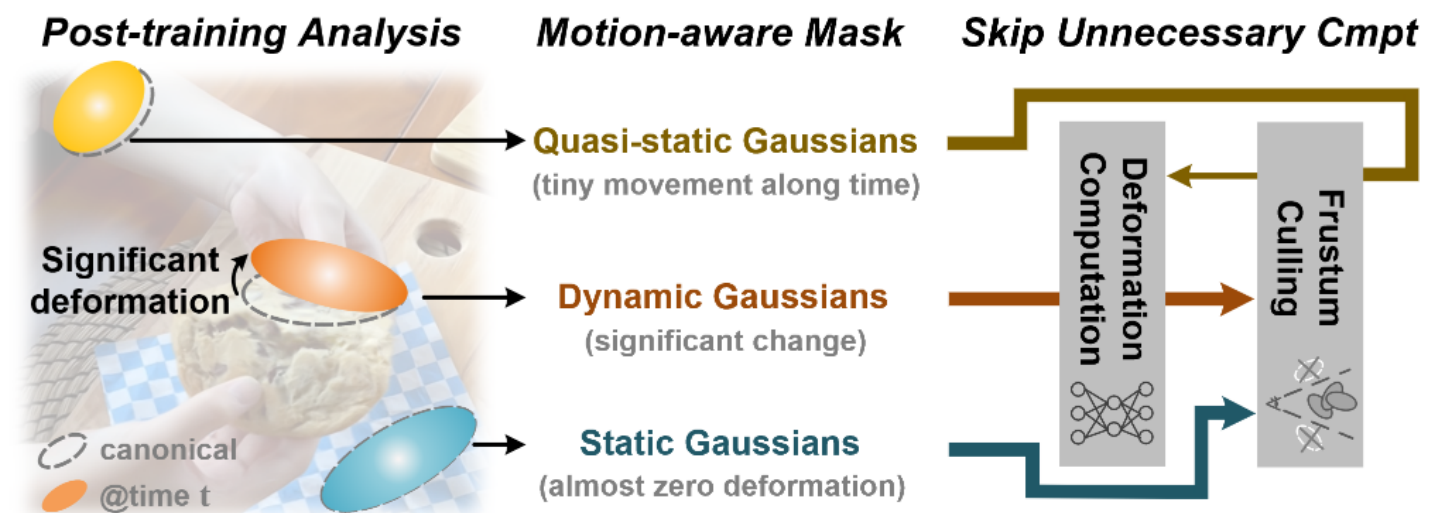


IV. Algorithm optimization

1. 后训练静态分析区分动静高斯球（三级标签：静止-微动-巨变）

训练结束后，离线分析该场景中各高斯球在整个时间跨度上的历史最大形变（中心点坐标），并对他们进行排序。按照一定的比例划分静止-微动-巨变三类高斯球（例如40%，40%，20%）。并且按类别连续存在主存，使得在形变场计算和预处理阶段可以有更规整的memory access。在类的内部再按照空间相近的连续存储。

该标签最小值需要2bit，因此对于存储上增加的开销可以忽略不计（原本每一个静态高斯球需要59个浮点数参数）。另外，由于三种类别的高斯球会分别连续存储，所以其实2bit标签都是不必要的，可以直接记录区分发起始id。例如假定先存静止，再存微动，最后存巨变，只需记录微动高斯的第一个id1，巨变高斯的第一个id2即可。这样的话0~id1-1就是静止高斯球，id1~id2-1就是微动高斯，其余为巨变高斯。



2. 跳过所有不必要的形变场计算

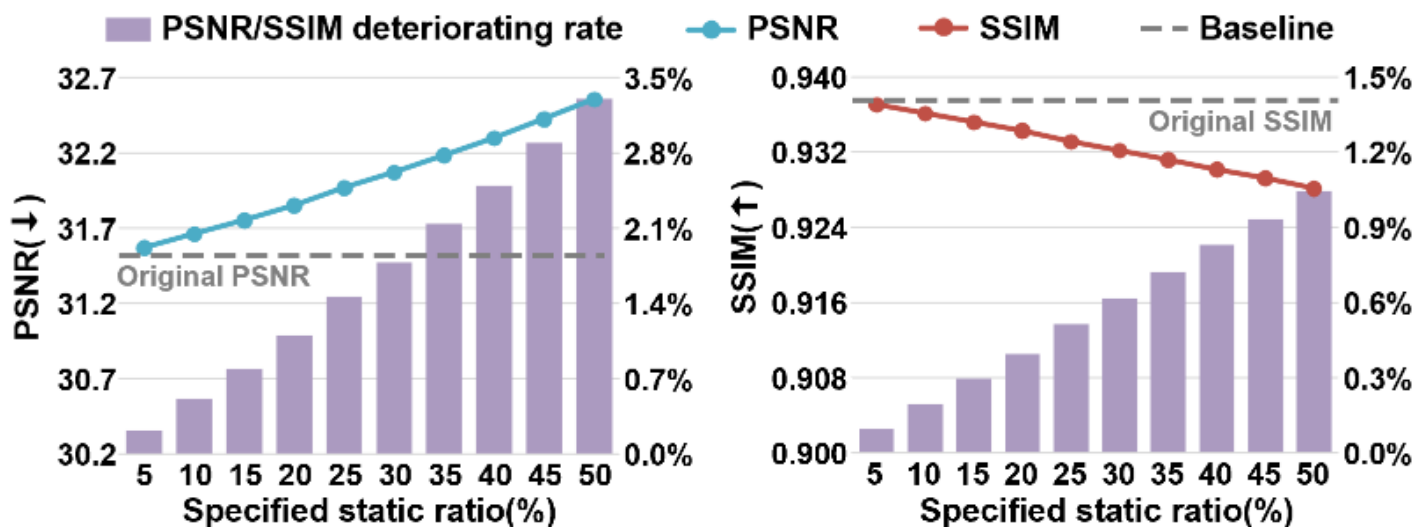
在完成训练后的动静标签后，渲染（推理）阶段，

- 对于静止高斯，任何时间步的形变计算都将跳过；
- 对于微动高斯，先做视锥剔除，再算未被剔除的高斯球的形变；
- 对于巨变的高斯，先算形变，再算是否在视锥内（是否需要剔除）。

这里还会具体通过实验证明跳过的形变场计算不会造成明显渲染质量下降，是对于两类高斯球（静止和微动）的跳过行为分别进行了证明。

静止高斯完全跳过形变场

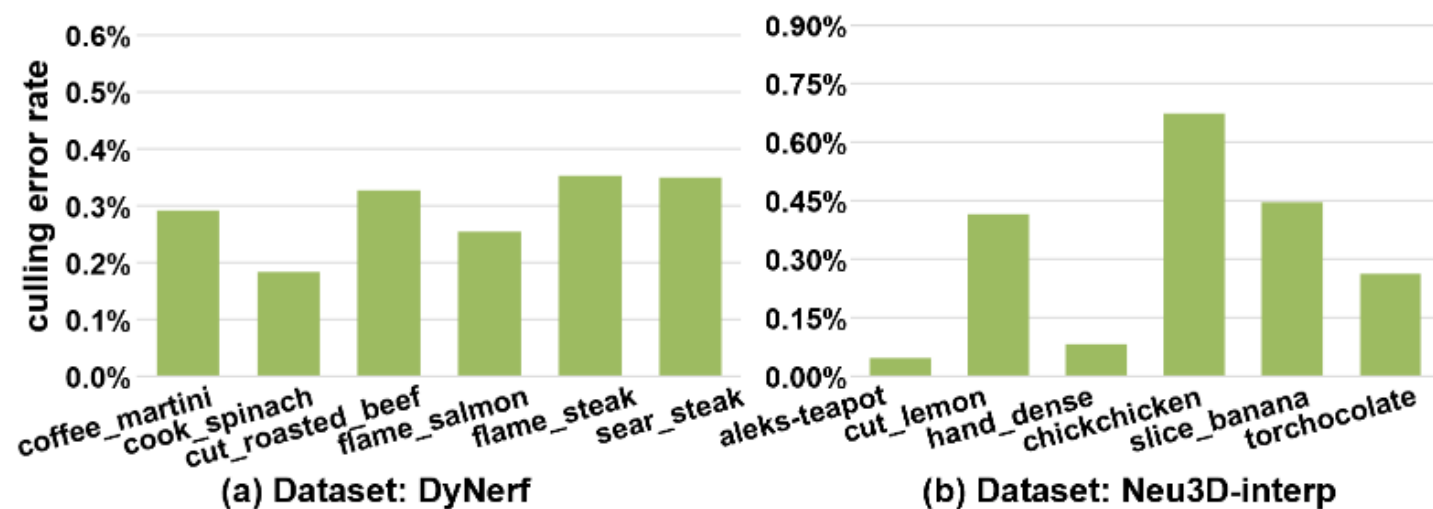
通过实验证明将一定比例的高斯球视作静止而跳过形变场，对于最终渲染指标PSNR/SSIM的恶化影响较小。



微动高斯球先剔除后形变

实验显示，那些最大形变最小的前90%高斯球实行不形变直接视锥剔除而导致的高斯球错误剔除率都在0.7%以下，因此对于渲染质量的影响极小。

此外，由于跳过形变场而直接剔除所引入的剔除错误都只在画面边缘产生，结合foveated rendering，本身在人眼注视点外围（通常就是画面边缘），可感知到的分辨率就会急剧降低，因此这边引入的误差是完全可以接受的。



3. 多分辨率光栅化（foveated rendering）

根据A3FR: Agile 3D Gaussian Splatting with Incremental Gaze Tracked Foveated Rendering in Virtual Reality以及Foveated 3D graphics等相关论文的foveated rendering方案，这里采取一种极为保守的视点外围降采样方案，即

中心区（0°~18°）：原始分辨率

Xin et al. (2025) 引用了 Lin et al. 的研究，指出在现代VR应用中，考虑到眼动追踪的误差（System Latency & Error），18°是一个保证“绝对安全”的高分辨率区域阈值。Guenter (2012) 的保守设置通常也在10°—15°左右，考虑到4DGS的动态伪影，放宽到18度是合理的。

过渡区（18°~30°）：降采样两倍（像素值减少1/2）

Patney et al. (2016) 指出，在应用对比度增强后，可以在 10° 以外就开始降采样，但为了平滑过渡，避免“隧道效应”（Tunnel Vision），保留一个过渡带是必要的。Fan et al. (2025) 也采用了类似的中间层级概念。

外围区（ $30^\circ \sim 90^\circ$ ）：降采样四倍（像素值减少3/4）

人类视觉在 30° 以外的分辨率急剧下降。Patney et al. (2016) 证明在 30° 以外，检测敏锐度（Detection Acuity）远高于分辨敏锐度（Resolution Acuity），意味着只要保留低频信息（如运动和亮度），高频细节（高分辨率）是完全不可见的。

基于Tile的稀疏渲染（Tile-based Sparse Rendering）

不同于传统光栅化的Foveated Rendering，针对3D Gaussian Splatting，我们采用了 **Xin et al. (2025)** 验证过的 **Adaptive Mesh Refinement (AMR)** 策略。

参数	设定值	参考文献支持	核心理由
Foveal Radius	18°	[Xin et al., 2025]	覆盖眼动误差，保证绝对清晰区。
Periphery Start	30°	[Patney et al., 2016]	分辨敏锐度 vs 检测敏锐度的分界点。
Sampling Method	Tile-based Sparsity (AMR)	[Xin et al., 2025]	契合3DGS的Tile-based光栅化流程。
Downsample Factors	1x / 2x / 4x	[Guenter et al., 2012]	符合MAR线性衰减模型，且保留了安全裕量。

（但是这里只规定了区域划分的视角。实际渲染以tile为单位，到底哪些tile是需要降采样的，还需取决于人眼到屏幕的距离d）

降采样2x意味着每两个像素只计算一个像素的光栅化，另一个像素根据附近的像素进行双线性插值得到颜色。

4. 均衡负载&访存友好schedule方案（Windowed Spatial-Locality Aware Scheduling）

由于foveated rendering在光栅化阶段会对不同的tile进行不同程度的降采样，从而进一步加剧了tile负载不均衡，假如按照普通的几何顺序进行贪婪分配，可能导致长尾效应，在多核情况下尤为严重，导致资源利用率不高，latency增加。如果单纯考虑workload，使用LPT策略，会打乱空间顺序，导致相邻的Tile被分发到时间上相距很远的位置，Cache/SRAM命中率极低，DRAM带宽爆炸。

因此本文提出一种光栅化阶段使得具有多个光栅化单元即多核系统进行负载均衡的调度，并且考虑了访存的规整性即数据局部性，从而尽可能提高硬件利用率以及降低光栅化阶段用时、降低dram带宽需求。

问题定义

M个tile，N个光栅化单元。每个tile的workload（在没有开始光栅化前）近似建模为该tile上交叉的高斯球。用时定义为所有M个tile全部光栅化完成的时间。现在需要在考虑dram访存效率的前提下，找到一种用时最短的调度方案。

方案设计

滑动窗口 + 希尔伯特曲线 + 局部LPT。

这个方案的核心在于：**在宏观上保持空间连续性（为了DRAM），在微观上进行贪心调度（为了负载均衡）。**

- 空间重排 (Spatial Reordering): 将M 个 Tile 按照 **希尔伯特曲线 (Hilbert Curve)** 或 **Z-order (Morton Code)** 进行排序。得到一个有序队列 $Q_{spatial} = [Tile_{H1}, Tile_{H2}, \dots, Tile_{HM}]$
- 滑动窗口 (Sliding Window / Look-ahead Buffer): 在调度器 (Scheduler) 中引入一个大小为 K 的 **滑动窗口 (Instruction Window / Reorder Buffer)**。K 的取值通常是N的倍数。调度器每次只从 $Q_{spatial}$ 中取出前K个 Tile 放入窗口。只有窗口内的 Tile 才有资格被分发。
- 窗口内贪心 (Intra-Window Greedy Dispatch): 当某个光栅化单元 RU_i 空闲时，查看窗口内这K个 Tile 的 Workload（即高斯球数量）。找出尚未被分发的 Tile。直接取**最重的**（Workload最大）Tile 给当前的空闲 RU（类LPT策略）。
- 窗口滑动 (Commit & Slide): 当窗口中最前端的 Tile 完成分发（或者完成处理，取决于具体实现复杂度）后，窗口向后滑动，吸入新的 Tile 进入候选池。

伪代码

代码块

```
1  Algorithm 1: Windowed Spatial-Locality Aware Scheduling (WSLAS)
2
3  1:  // Phase 1: Spatial Reordering (Pre-processing)
4  2:  Compute Hilbert curve index  $H(t)$  for each tile  $t \in \mathcal{T}$ 
5  3:  Sort  $\mathcal{T}$  in ascending order of  $H(t)$ 
6  4:  Initialize sliding window  $\mathcal{W} \leftarrow \emptyset$ 
7  5:
8  6:  // Phase 2: Dynamic Scheduling Loop
9  7:  while  $\mathcal{T} \neq \emptyset$  or  $\mathcal{W} \neq \emptyset$  do
10  8:
11  9:      // Step A: Refill Window (Preserve Spatial Locality)
12  10:      while  $|\mathcal{W}| < K$  and  $\mathcal{T} \neq \emptyset$  do
13  11:           $t_{next} \leftarrow \text{Dequeue}(\mathcal{T})$ 
14  12:           $\mathcal{W} \leftarrow \mathcal{W} \cup \{t_{next}\}$ 
15  13:      end while
16  14:
17  15:      // Step B: Dispatch to Idle Units (Greedy Load Balancing)
18  16:      for each unit  $r \in \mathcal{R}$  do
19  17:          if  $r$  is IDLE and  $\mathcal{W} \neq \emptyset$  then
20  18:              // Select the heaviest tile within the local window
21  19:               $t_{best} \leftarrow \arg\max_{t \in \mathcal{W}} (\text{Workload}(t))$ 
22  20:
23  21:              Dispatch  $t_{best}$  to unit  $r$ 
```

```

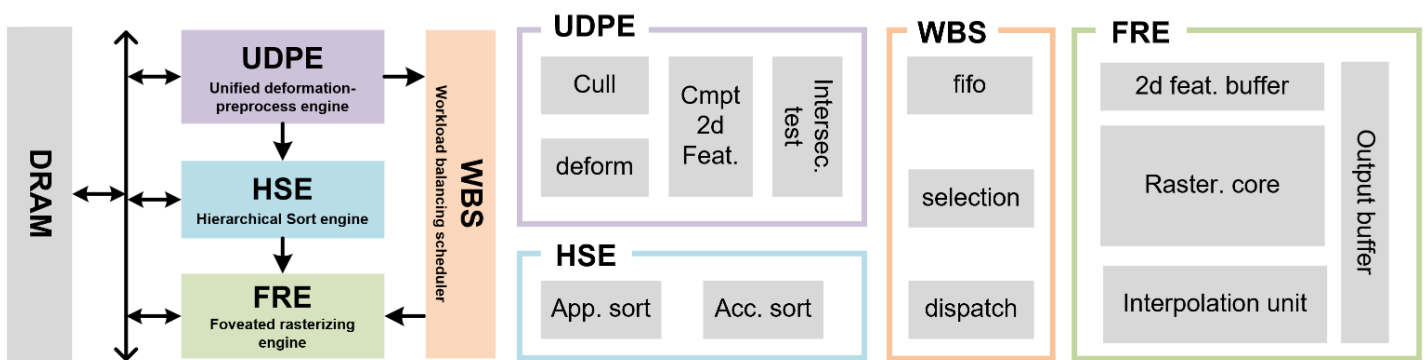
24  22:          $\mathcal{W} \setminus \rightarrow \mathcal{W} \setminus \setminus \{t_{best}\} \setminus$
25  23:          end if
26  24:      end for
27  25:
28  26:      Wait for next clock cycle or event
29  27: end while

```

V. Architectural design

1. 整体架构+数据流

架构（草图）



数据流

- **DRAM**: canonical高斯球按静止-微动-巨变的顺序按类连续存储，每个类里面顺序按照空间相邻顺序存储。并且每类参数分别连续存储，即 $a_1, a_2, a_3, \dots, b_1, b_2, b_3, \dots$ 而非 $a_1, b_1, c_1, \dots, a_2, b_2, c_2, \dots$ （ a, b, c 为不同类型的参数例如中心点坐标、四元数等）。另外再记录静止-微动-巨变分隔处的id，从而标识哪一段是静止，哪一段是微动，哪一段是巨变。
- **第一步**：高斯球按照不同的类别，送入预处理模块（UDPE）。cull模块获取目标高斯球的中心点坐标并进行相机视角变换，然后进行视锥剔除，并且将判断结果按照id顺序存回dram，形成一个valid数组。而deform模块则也是获取目标高斯球的中心点坐标并结合当前帧给定的时间戳进行形变场计算，将加上形变量后的高斯球参数按照id顺序写回dram，形成针对当前帧的高斯球参数数组。最后计算二维协方差矩阵以及颜色等2d features。
- **第二步**：intersection test。需要根据中心点坐标跟协方差矩阵跟gscore一样，进行subtile粒度的交叉测试，并且生成每个tile上相交的高斯球id: depth键值对列表。这个交叉测试是逐高斯进行的因此也是全局的，可以算是预处理步骤的一部分。交叉结果得出后就可以送入WBS用于规划调度，并将当前目标tile优先送入HSE进行排序。
- **第三步**：在HSE中进行tile内深度排序。跟gscore一样第一轮使用pivot进行大致的排序（或者说partition）分成不同的chunk，然后第二轮在chunk内进行精确排序（双调排序）。这一阶段已经是各tile之间独立进行了，因此需要分配多个sorting单元分别负责不同的tile，而且需要和下一步的光栅化紧密相接，因此最好是一个排序单元对应一个光栅化单元。每个chunk排序结果是一个有序id列表，最好是能存在片上，如果存不下则存回dram。

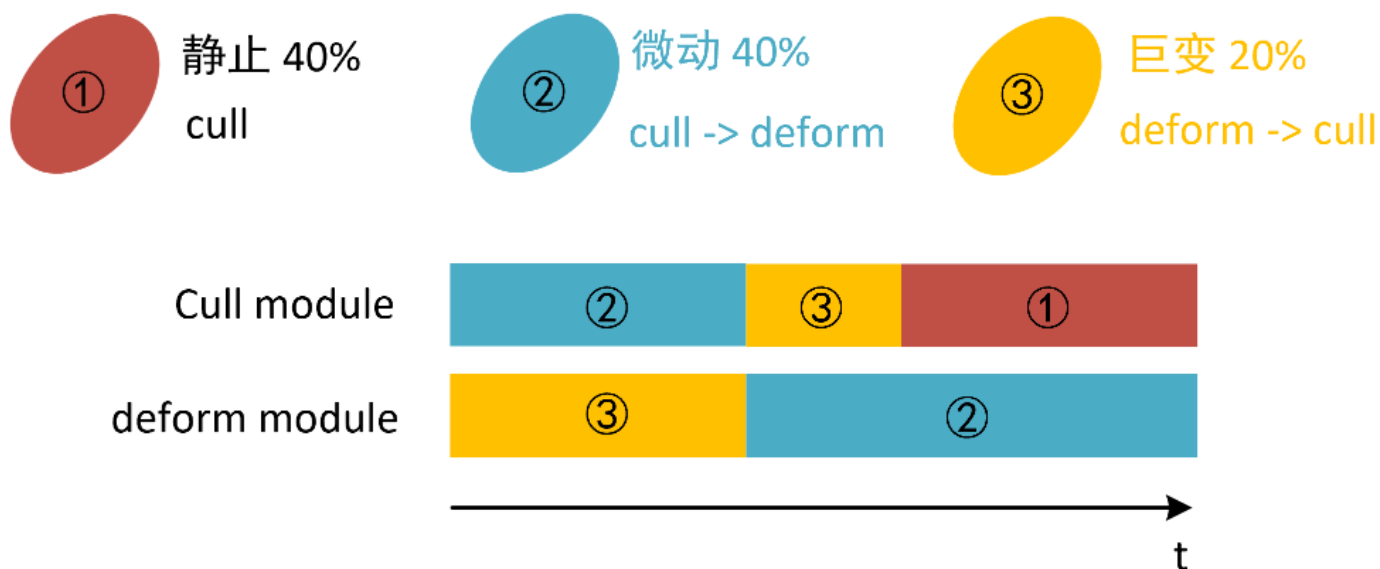
- 第四步：一旦上一步有一个chunk的精确排序完成，就可以开始光栅化了。每个光栅化core去根据排序结果取出对应id高斯球的参数，然后多个像素并行计算alpha-blending。这期间负载分配由调度器完成，考虑了利用空间局部性的负载均衡策略。那些需要插值计算的像素只要周围的像素被计算完成就开始插值计算。最后将计算完的像素颜色写回dram，全部完成后就算该帧渲染完毕。

2. 视锥剔除-形变计算cross stage并行预处理模块 (unified deform-preprocess engine)

不同类别（静止-微动-巨变）穿插处理视锥剔除-形变计算提高子模块利用率，合起来作为一个预处理模块。

为了支持灵活的数据流向，硬件前端需要一个 **标签解析与分发器 (Label Parser & Dispatcher)**。

- **读取与解析**：在获取高斯球数据时，Dispatcher 首先读取其离线分析生成的 2-bit 动静标签。注意，所有的高斯球的动静标签应该按照id顺序连续存储，也就是说对于一个10万高斯球的场景，需要连续的一段2
- **动态路由 (Dynamic Routing)**：根据标签类别，Dispatcher 通过多路选择器 (MUX) 和交叉开关 (Crossbar) 将高斯球的数据流引向不同的硬件路径：
 - **静止 (Static, 40%)**：直接送入 Cull Module
 - **微动 (Quasi-static, 40%)**：先送入 Cull Module
 - **巨变 (Dynamic, 20%)**：先送入 Deform Module



为了支撑上述穿插调度，预处理模块内部需要以下关键微架构设计：

- **跨阶段异步 FIFO (Inter-Stage FIFOs)**：在 CM 和 DM 之间必须插入足够深度的 FIFO。因为视锥剔除的通过率是动态变化的（受视角影响），Phase 1 中微动高斯球通过 CM 的速率可能与 DM 处理巨变高斯球的速率不匹配。FIFO 可以吸收这种非确定性的流量波动 (Traffic Fluctuation)，防止流水线气泡 (Pipeline Bubbles)。

- **零开销旁路逻辑 (Zero-Overhead Bypass Logic)**: 对于 40% 的静止高斯球 (), 在 CM 完成剔除后, 硬件会通过旁路数据通路 (Bypass Datapath) 直接将其送入后端的排序和光栅化阶段, 彻底关闭 (Clock/Power Gating) 其在 DM 中的算力消耗, 实现最大化的功耗收益
- **统一坐标更新器 (Unified Coordinate Updater)**: 在模块末端, 需要一个合并单元。它接收来自 CM (静止高斯球)、从 CM 经 DM (微动高斯球)、以及从 DM 经 CM (巨变高斯球) 的三路数据。该单元负责将原始 Canonical 坐标与经过 DM 算出的修正量 $\Delta\mu$ 相加, 输出最终的 t 时刻有效 3D 坐标, 为后续的 Gaussian Sorting 做好准备。

3. 多分辨率的注视点渲染光栅化模块 (foveated rasterization engine)

GS Core 是 4 个 core, 每个 core 负责一个 tile (16x16), core 内 8x8 个 VR, 每个 VR 负责一个 subtile (2x2 像素)。

(包括插值计算功能)

注意以下的参数只是初步设计, 在 simulator 中应全部设为可调参数, 并且通过实验得到一个最佳的参数组合。

但是由于 foveated rendering 会有 1/2 和 1/4 降采样, 因此对于 4x 降采样的情况应使得一个 core 可以渲染一个 tile, 那么无降采样的 tile 则是由 4 个 core 计算或者由 1 个 core 经历 4 轮计算完成。这样可以使得所有计算单元都不要空闲, 并且可以更加细粒度地调控。

现在初步规定: 每个 core 负责 32x32 的 tile, 每个 core 可同时计算 256 个像素, 即具有 8x8 个 VR, 每个 VR 负责一个 subtile (2x2)。因此对于一个 4x 降采样的 tile, 实际渲染 256 个像素, 刚好可以由一个 core 负责, 而对于无降采样的 tile 需要渲染 1024 个像素, 可以视情况让 4 个 core 同时处理一轮或者 1 个 core 算 4 轮。因此从 core 的角度来说, 所有 tile 都将被划分为 **256 像素的工作组 (Work Group, WG)** 为基本单元的任务。

整个多分辨率光栅化模块主要由三个子单元构成: **自适应任务分发器 (Adaptive Tile Dispatcher)**、**统一体积渲染阵列 (Unified VR Array)** 和 **插值重建引擎 (Interpolation & Reconstruction Engine)**。

- **Adaptive Tile Dispatcher**: 接收来自调度器 (Scheduler) 的 Tile 任务。根据视点到该 Tile 的偏心角 (Eccentricity Angle), 确定其降采样等级 (1x, 2x, 4x)。它将一个 32x32 的逻辑 Tile 拆分为若干个 **256 像素的工作组 (Work Group, WG)**。
- **Unified VR Array**: 包含 64 个 Volume Rendering (VR) Units, 组织成 8x8 的阵列。每个 VR Unit 内部包含 4 个并行的流水线 (Lane), 负责 4 个像素点的 α 计算、透射率更新 (Early Termination) 和颜色混合 (Color Blending)。该阵列的单周期吞吐量严格锁定为 256 个像素。
- **Interpolation & Reconstruction Engine**: 位于流水线末端。当收到 2x 或 4x 降采样的 Tile 渲染结果时, 利用双线性插值 (Bilinear Interpolation) 将稀疏像素还原为 32x32 的完整分辨率块, 然后写回 Frame Buffer。该单元由轻量级的移位寄存器和加法器树组成 (例如, 两像素求均值仅需一次加法和一次右移 `>> 1`), 由于不涉及乘法器, 其面积开销极小。硬件会在缓存中保留当前渲染的边界像素, 以消除跨 Tile 插值时产生的接缝突变 (Tunnel Vision)。

4. 负载均衡调度器 (workload balancing scheduler)

Scheduler 位于 **Tile Generator (前端几何处理)** 和 **Rasterization Units (后端光栅化阵列)** 之间。

主要包含三个核心子模块：

1. **Instruction Window (IW) / Issue Queue**: 存储当前待调度的 K 个 Tile 的信息（寄存器堆结构）。
2. **Selection Logic (Arbiter)**: 纯组合逻辑，用于从窗口中通过比较器树（Comparator Tree）快速选出 Workload 最大的 Tile。
3. **Dispatch Logic & Scoreboard**: 维护 RU 的状态，并控制数据通路的选通。

A. 输入缓冲 (Ingress FIFO)

- **功能**: 接收来自上游（经过希尔伯特排序）的 Tile 流。
- **结构**: 标准同步 FIFO。
- **目的**: 解耦上游处理速度和 Scheduler 的分发速度。

B. 指令窗口 (Instruction Window - The "Pool")

这是整个 Scheduler 的心脏。

- **存储介质: Register File (寄存器堆)** 而非 SRAM。
 - *原因*: 我们需要在一个周期内**并发读取**所有 K 个 Entry 的 Workload 值进行比较。SRAM 无法支持这种并发访问。
- **Entry 结构 (Slot Definition)**:
- 每个 Slot 包含以下寄存器字段：
 - **Valid (1-bit)**: 该槽位是否有有效 Tile。
 - **Tile_ID (x-bit)**: 指向 Tile 数据（可能存储在较大的 SRAM Buffer 中）的指针。
 - **Workload (y-bit)**: 高斯球数量（用于比较）。
 - **Age (z-bit)** (可选): 防止饿死 (Starvation)，如果一个任务虽然小但等太久了，也可以强制提升优先级。

C. 选择逻辑 (Selection Logic - The "Brain")

这是一个纯组合逻辑电路，负责在单周期内找到“最佳候选者”。

- **核心组件: 比较器树 (Comparator Tree / Reduction Tree)**。
- **逻辑流程**:
 - a. 输入 K 个 Workload 值（仅当 Valid=1 时有效）。
 - b. 第一级比较：比较 Slot[0] vs Slot[1], Slot[2] vs Slot[3]...
 - c. 逐级向上冒泡，每一级输出较大的那个 Workload 及其 Index。

d. 最终输出: `Winner_Index` 和 `Winner_Workload`。

- **关键路径:** 这是时序最紧张的地方。如果 $K=16$ ，树的深度为 4 级 ($\log_2 16$)，加上 MUX 延迟，通常能跑在很高频率。如果 K 很大（如 64+），可能需要流水线化。

D. 计分板与分发 (Scoreboard & Dispatch)

- **RU Status Table:** 维护 N 个光栅化单元的状态 (IDLE / BUSY)。
 - 使用 `One-Hot` 编码或 `Priority Encoder` 快速找到第一个 IDLE 的 RU。
- **Dispatch Action:**
 - a. 当 `Selection Logic` 选出 `Winner_Index` 且有 RU 处于 IDLE 状态。
 - b. 触发分发信号: 将 `Winner_Index` 对应的 `Tile_ID` 发送给目标 RU。
 - c. **Clear Operation:** 在下一个时钟沿，将 Instruction Window 中 `Winner_Index` 位置的 `Valid` 置为 0（腾出空位）。

VI. Evaluation

1. Setup

- benchmarks (dataset) : dynerf, hypernerf
- baselines: Jetson, GScore
- Software configuration: img pixel (1440x1024) , tile size (32x32) , subtile size (2x2) ,
- Architectural configuration:

2. Hardware implementation

使用 tsmc22nm 或者实验性 7nm 的 pdk 综合得到面积和功耗;

为了得到延迟编写了 cycle accurate simulator。

3. Performance

- 纯软件端加速的性能结果: 使用跳过形变+注视点渲染两种技术的加速结果 (统一使用 jetson, 原算法为 baseline)
- 渲染质量评估: 使用跳过形变场的一套最终参数下的渲染指标 SSIM/PSNR 完整数据+使用注视点渲染前后的两张图片对比使人肉眼感觉差距较小 (没有明显下降)
- 软硬件总方案的加速效果: 使用算法优化 (跳过形变&注视点渲染)+调度+asic, 相比于 jetson 和 gscore 的帧率对比
- 软硬件总方案的功耗减少效果: 整套方案 (软硬件) 相比于 jetson (baseline) 的面积效率对比

VII. Conclusion

